



Lớp Money — magic methods `__str__` + `__eq__`

Cài đặt class Money biểu diễn số tiền (amount + currency). Cần overload **phép in** ra dạng "amount currency" và **phép so sánh bằng** — 2 Money bằng nhau khi cùng amount **VÀ** cùng currency.

Python: override `__str__` + `__eq__`. `print(m)` tự gọi `__str__`, `m1 == m2` tự gọi `__eq__`.

C++: overload friend `std::ostream& operator<<` và `operator==` (free function hoặc friend để truy cập private fields). `std::cout << m` gọi `operator<<`; driver in "True"/"False" từ kết quả `m1 == m2`.

Bạn chỉ nộp khai báo lớp

File .py bạn nộp chỉ chứa **Money** — **không** viết main, if `__name__`, hay driver đọc stdin. Grader **lấy tên file bạn nộp** làm tên module, **import lớp** của bạn, **chạy driver** để ném stdin vào và so sánh stdout.

 **Đặt tên file** là tên identifier hợp lệ (vd. `Point.py`, `TuLanh.py`, `Money.py`). Nếu đặt bậy (my-bai.py, 1.py) grader fallback `student_solution`.

Luồng chấm bài

Money.py (bạn nộp) → from Money import * → driver grader → stdin → stdout → so khớp = pass

 Khung lớp bạn nộp lên

PythonC++

```
class Money:

    def __init__(self, amount: int, currency: str = 'VND'):

        self.amount = amount

        self.currency = currency

    # TODO: __str__ → 'amount currency' (vd '100000 VND')

    # TODO: __eq__ → True khi cùng amount VÀ cùng currency
```

```

class Money {

    long amount_;

    std::string currency_;

public:

    Money(long a, const std::string& c = "VND") : amount_(a), currency_(c) {}

    // TODO: operator<< (ostream&, const Money&) – in 'amount currency'

    // TODO: operator== (const Money&, const Money&) – so sánh cả 2 trường

    // Hint: khai báo friend để access private fields.

    long amount() const { return amount_; }

    const std::string& currency() const { return currency_; }

};

// TODO: std::ostream& operator<<(...) và bool operator==(...)

```

□ Grader sẽ làm gì với lớp của bạn

1. Đọc 2 dòng `amount currency` (vd `100000 VND`)
2. Tạo `m1, m2`
3. In `m1` rồi `m2` — Py: `print(m1)` (gọi `__str__`) / C++: `std::cout << m1` (gọi `operator<<`)
4. So sánh `m1 == m2` — Py: `print(m1 == m2)` in `True/False` / C++: driver tự in `"True"/"False"` dựa trên kết quả `operator==`

► Xem code driver grader tự chạy

Grader chạy đoạn code sau cùng file bạn nộp. Dùng nút ở khung khai báo để xem driver cho từng ngôn ngữ:

```

from Money import * # file SV nộp

line1 = input().split()

```

```

m1 = Money(int(line1[0]), line1[1])

line2 = input().split()

m2 = Money(int(line2[0]), line2[1])

print(m1)

print(m2)

print(m1 == m2)

```

```

#include <iostream>    // grader tự prepend khi compile

```

```

// ... + class bạn viết ...

```

```

int main() {

    long a1, a2; std::string c1, c2;

    std::cin >> a1 >> c1 >> a2 >> c2;

    Money m1(a1, c1), m2(a2, c2);

    std::cout << m1 << std::endl;

    std::cout << m2 << std::endl;

    std::cout << ((m1 == m2) ? "True" : "False") << std::endl;

    return 0;

}

```

□ Testcase mẫu — để đối chiếu format

Grader chạy *code SV + driver* rồi so sánh stdout từng dòng (strip whitespace cuối, bỏ dòng trống thừa ở cuối). Các testcase ẩn có cùng định dạng.

Ví dụ 1

stdin

```
100000 VND
```

```
100000 VND
```

expected stdout

```
100000 VND
```

```
100000 VND
```

```
True
```

Ví dụ 2

stdin

```
500 USD
```

```
500 VND
```

expected stdout

```
500 USD
```

```
500 VND
```

```
False
```

Ngôn ngữ hỗ trợ: bài này nhận `.py` / `.cpp`. Grader tự ghép driver + class bạn nộp rồi chạy. Java sẽ mở khi có driver tương ứng.

□

Đây là bài **thực hành** — bạn **nộp lại bao nhiêu lần cũng được**. Mỗi lần nộp, hệ thống giữ điểm cao nhất. Thử, đọc error, fix, thử tiếp.